

Checkpoint weight averaging in GoLLeM-v5: a test at a high learning rate and a rule for the end of training

Arkadiusz Słota*

27 September 2026 — short note, version 1 (64M run complete; 128M run still training)

Abstract

We average the weights of three consecutive checkpoints of the same training run and evaluate the result as a new model, without any further training. On the final 64M GoLLeM-v5 run, at a high learning rate (83–91% of peak), averaging three checkpoints 20k steps apart improved our selection score by +0.36 and +0.43 eff over the last of the three checkpoints, mostly through BLiMP and WikiText, and no axis got worse by more than its noise. Both the decision rule and its thresholds were written down before the numbers were computed, and all numbers come from a selection split, not from the board test set (for BLiMP, which has no separate split, the selection half is also half of the board’s BLiMP score; see Section 2). A second rule, written in our internal team log before the result, decided whether the average of the last three checkpoints of the final runs (720k, 740k and 760k steps, learning rate $\approx 10\%$ of peak) replaces the last checkpoint as the submitted model. For the 64M run the average gained +0.195 eff on the selection axis, below the 0.3 threshold, so the 760k checkpoint was kept (board-protocol eff 76.07; the average, measured only after the choice and only for the record, 76.20). At the end of training the gain is about half of the gain at a high learning rate, as the reasoning predicts. The 128M run is still training; its outcome will be added in version 2. This is a small study: one model size so far and one run.

1 What we do

Given checkpoints $\theta_{k_1}, \dots, \theta_{k_n}$ of one run, the averaged model is

$$\bar{\theta} = \frac{1}{n} \sum_{i=1}^n \theta_{k_i},$$

computed tensor by tensor in float32 over every floating-point tensor of the state dictionary (62.9M unique parameters for the 64M model; the input embedding and the output head share one matrix, so averaging both copies gives the same shared tensor). Non-floating-point tensors and the configuration are taken from the last checkpoint; the checkpoints of this run contain only float32 tensors, so the earlier version of our tool used for the test in Section 3, which averaged every tensor, produces a bit-identical model. Nothing is trained: the average is written as an ordinary checkpoint and evaluated with the same protocol as any other. It takes seconds on a CPU.

Why it can help. With a high learning rate, successive checkpoints move around a region of low loss rather than settling in it; each step adds some noise from its batch. An average of nearby points of such a trajectory can lie closer to the centre of the region than any single one of them. This is the idea behind Polyak averaging, stochastic weight averaging (SWA) and latest-weight averaging (LAWA); see Section 6. The same reasoning predicts that the gain shrinks when the learning rate is low, because checkpoints then lie close to each other.

*Prepared with the Kolektyw AI agents: Hart (experiments and numbers), Latarnik (draft and editing), Wartownik (review), Monter (training infrastructure).

2 Setting and measurement

Model and run. The final 64M GoLLeM-v5 run: 62.9M parameters, 14 layers, width 576, a cosine learning-rate schedule over 760k steps from the peak to 10% of it, the same data pool (9.39B tokens) and trainer for the whole run. Checkpoints are saved every 20k steps with the optimiser state and published on the Hugging Face Hub (SlayerLab/gollem-v5-ckpts).

Selection axis. Decisions use a selection split: ARC-Easy validation and training questions ($n = 2783$), one half of the BLiMP pairs ($n = 33,500$) and WikiText validation byte perplexity, with a fixed exclusion list of 38 ARC questions and 8 WikiText articles (a frozen list from our scans of the training pool). The ARC-Easy and WikiText parts are disjoint from the board test sets. BLiMP has no separate split, so the selection half is also half of the pairs on which the board scores BLiMP; the board test score is never used for a decision. The score is the board formula applied to these numbers,

$$\text{eff} = \frac{1}{3}(\text{ARC} + \text{BLiMP} + \text{WS}) \cdot m(P), \quad \text{WS} = 100 \left(1 - \frac{\ln(w/1.86)}{\ln(500/1.86)} \right),$$

where w is the byte perplexity and $m(P) = 1.03646$ is the parameter multiplier for 62.9M parameters. The evaluation is deterministic.

Noise thresholds. The thresholds σ used in the rules were fixed in advance, not estimated from this test: $\sigma_{\text{ARC}} = 1.0$ point, about one binomial standard error at $n \approx 2.8k$; $\sigma_{\text{BLiMP}} = 0.35$ points, above the binomial standard error of about 0.24 at $n = 33,500$; and $\sigma_w = 0.002$ in byte perplexity, a conservative value chosen by us for which we have no independent estimate on this split.

3 The test: a high learning rate

Rule, fixed before computing. Two arms, each an average of three checkpoints 20k steps apart, compared with the last checkpoint of its window: S1 = {120k, 140k, 160k} against 160k, and S2 = {180k, 200k, 220k} against 220k. Averaging “helps” only if *both* arms beat their reference by more than 0.3 eff on the selection axis *and* no axis of either arm is worse than its reference by more than its σ .

model	LR (% of peak)	ARC-E	BLiMP (half)	wiki byte ppl	eff	Δ
single 160k	91	44.16	73.85	2.4389	73.65	
S1: mean of 120k, 140k, 160k		43.77	75.24	2.4317	74.01	+0.36
single 220k	83	44.48	74.56	2.4154	74.06	
S2: mean of 180k, 200k, 220k		44.66	75.43	2.3903	74.49	+0.43

Table 1: The test, selection axis only. eff to two decimals; unrounded: 73.646, 74.010, 74.062, 74.489 (recomputed independently from the result files).

Outcome. Both arms passed the threshold ($+0.364$ and $+0.427 > 0.3$; S1 by a margin of only 0.064). No axis fell by more than its noise: in S1 ARC-Easy fell by 0.39 ($\sigma = 1.0$) while BLiMP rose by 1.39 and byte perplexity fell by 0.0072; in S2 all three axes improved (ARC-Easy $+0.18$, BLiMP $+0.87$, byte perplexity -0.0251). The gain is carried by BLiMP and WikiText, not by ARC-Easy. This is one model at one stage of training.

4 A rule for the end of training

Rule. At the end of the run the learning rate is about 10% of peak (10.6% at 720k, 10.0% at 760k), so checkpoints lie close together and the test need not transfer. On 27 September 2026 at 00:57 UTC, with the 64M run at step 300k and the 128M run at step 200k, we fixed the following rule for both final runs:

1. The main reported result is the checkpoint at step 760k; training is not changed.

2. The average $\bar{\theta}$ of the checkpoints at 720k, 740k and 760k is computed exactly as in Section 1.
3. Both are evaluated on the selection axis with the protocol of Section 3.
4. One model per size is submitted to the board: the average if its selection eff exceeds that of 760k by more than 0.3 and no axis is worse than 760k by more than its σ ; otherwise the 760k checkpoint.
5. Board-test numbers for both models are computed only after this choice, and both rows are reported.

The rule was written in our internal, timestamped team log and was not published before the result; this note was released after the 64M outcome below.

Outcome for 64M. The run finished at step 760k on 27 September 2026 at 19:41 UTC. On the selection axis the single checkpoint scored 75.764 (ARC-Easy 47.22, BLiMP half 76.22, byte perplexity 2.3452) and the average 75.959 (47.54, 76.41, 2.3382): a gain of +0.195, with all three axes slightly better, below the +0.3 threshold. The rule therefore chose the **760k checkpoint**. Only then were board-test numbers computed (our evaluation loop was paused before step 760k so that no board number existed before the choice): 760k checkpoint **76.07** eff (ARC-Easy 48.19, BLiMP 76.16, byte perplexity 2.3474); the average, for the record only, 76.20 (48.36, 76.31, 2.3406). The gain at a learning rate of about 10% of peak (+0.195 on the selection axis) is about half of the gains at 83–91% of peak (+0.36, +0.43), in line with Section 1 and with the observation of Sanyal et al. [5].

Outcome for 128M. The 128M run is still training (expected end: 28 September). The same rule applies.

5 What this is and what it is not

- It is not tuning on the test set: decisions use only the selection split, the rules were written before the numbers, and board-test numbers are read only after the choice. For BLiMP the selection half is part of the board’s pairs (Section 2), so the protection there is the pre-registered rule, not disjointness.
- It costs no training compute and no data: only disk space for the checkpoints, which a long run keeps anyway.
- It is not a result for other sizes: the test used the 64M run only. The rule of Section 4 covers the 128M run as well.
- The end of training was measured, not assumed: for the 64M run the gain there was about half of the gain at a high learning rate and too small to change the submitted model.

6 Related work

Averaging the iterates of stochastic optimisation goes back to Polyak and Juditsky [1] and Ruppert [2]. Stochastic weight averaging (SWA) averages points along a trajectory with a high or cyclical learning rate [3]; latest-weight averaging (LAWA) averages the last few checkpoints [4], and for language-model pre-training its gains were reported to be larger with high learning rates [5], which agrees with the reasoning in Section 1; model soups average separately fine-tuned models [6]. We claim no new method. Our contribution is modest: a small test and a rule, both fixed in advance and judged on a selection split, on a small language model trained for a public leaderboard.

References

- [1] B. T. Polyak and A. B. Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4), 1992.
- [2] D. Ruppert. Efficient estimations from a slowly convergent Robbins–Monro process. Technical report, Cornell University, 1988.
- [3] P. Izmailov, D. Podoprikin, T. Garipov, D. Vetrov and A. G. Wilson. Averaging weights leads to wider optima and better generalization. *UAI*, 2018.

- [4] J. Kaddour. Stop wasting my time! Saving days of ImageNet and BERT training with latest weight averaging. arXiv:2209.14981, 2022.
- [5] S. Sanyal, A. Neerkaje, J. Kaddour, A. Kumar and S. Sanghavi. Early weight averaging meets high learning rates for LLM pre-training. arXiv:2306.03241, 2023.
- [6] M. Wortsman et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. *ICML*, 2022.

Acknowledgements

The experiments, measurements and this note were prepared together with the Kolektyw AI agents: Hart ran the tests, Wartownik recomputed every number and reviewed the text, Monter ran the training infrastructure, and Latarnik wrote the draft.